MessMate

Complete Frontend Implementation Prompt

UPDATED EDITION · React + TypeScript + Vite · FastAPI later · No Streamlit

Purpose

A ready-to-use specification for a coding agent to build the complete MessMate frontend before backend development. Covers student, vendor and administrator portals, the source menu, mock interactions, notification schedule validation, menu versioning, tests and completion criteria.

Important boundary: *This phase does not implement a real backend, institutional authentication, a real scheduler, validated forecasting, or WhatsApp delivery. These will be connected later.*

1. Product Objective

MessMate helps collect students' meal intentions, display the daily menu, provide final response counts to mess personnel, recommend preparation quantities, and eventually measure attendance and leftovers.

Roles: Student, Mess Vendor, Administrator. All use one login page; successful login redirects according to the account's role.

Student flow: receive notification → open meal → read menu → select intention → submit → change response before deadline if needed.

Vendor flow: open meal → review final counts → review recommended quantity → accept/edit quantity → confirm plan → record actual outcome.

Admin flow: manage users → manage menu → configure notification schedules → review reports.

2. Final Technology Stack — No Streamlit

Frontend: React + TypeScript, Vite, Tailwind CSS, shadcn/ui, Lucide React, React Router, Recharts, React Hook Form, Zod, date-fns.

Backend (later phase): Python + FastAPI.

Data and forecasting (later phase): SQLite initially, SQLAlchemy, Pydantic, Python, pandas and NumPy.

Testing: Vitest + React Testing Library for frontend; pytest for backend.

Tooling: ESLint, Git.

Hard constraint: Do not use Streamlit. Do not build a Python backend during this frontend phase. Keep business and forecasting logic in independent typed/mock service modules so the later FastAPI backend can replace them without rewriting UI components.

3. Visual Design System

Use approved Theme A: Modern & Minimal. Clean white and pale lavender backgrounds, deep purple primary colour, rounded cards, subtle borders/shadows, readable typography, restrained gradients, consistent spacing, and minimal clutter.

Suggested tokens: primary #6D35C5; dark purple #45208A; pale lavender #F4EEFF; background #FAF9FD; card #FFFFFF; main text #201A2D; secondary text #6B6577; success #21864A; warning #C88712; error #C83C4A; border #E7E0F1. Store centrally.

Support desktop, laptop, tablet, and mobile. On small screens collapse navigation, stack cards, keep response buttons large, prevent horizontal overflow, and keep deadlines/status visible. Make vendor actions especially simple.

Use semantic HTML, labelled inputs, keyboard navigation, visible focus states, sufficient contrast, accessible dialogs/menus, descriptive buttons, ARIA where appropriate, and reduced-motion support.

4. Frontend Architecture

Use a feature-oriented structure. Suggested folders: src/app (App.tsx, router.tsx, providers.tsx); src/components (ui, layout, feedback, navigation, common); src/features (auth, student, vendor, admin, menu, notifications, forecasting, attendance, reports); src/hooks; src/lib; src/services (api, auth, menu, notifications, responses, forecasting, reports); src/mocks; src/types; src/utils; src/styles; tests; public/images.

Include package.json, TypeScript/Vite/Tailwind/test/lint configuration, .env.example, .gitignore, favicon, and README. Do not put the entire app in one component. Separate pages, reusable components, mock data, types, and services.

5. Routing and Shared Layout

Public: /login, /forgot-password.

Student: /student, /student/meals, /student/meals/:mealId, /student/menu, /student/notifications, /student/history, /student/profile.

Vendor: /vendor, /vendor/planning, /vendor/meals/:mealId, /vendor/outcomes, /vendor/reports, /vendor/profile.

Admin: /admin, /admin/users, /admin/menu, /admin/menu/:menuId, /admin/schedules, /admin/reports, /admin/settings.

Create a not-found page, role-specific navigation and shared layouts. Route guards are prototype-only; real authorisation must be enforced by the backend later.

6. Shared Login

Create a split-screen login: left side with MessMate branding, tasteful food-related visual and short description; right side with institute email/ID, password, visibility toggle, login, forgot-password, validation and error messages.

Do not ask users to select their role. The mock auth service determines it. Provide documented fake demo accounts for student, vendor and admin. Support loading, invalid credentials, successful login, logout and session restoration. Do not claim institutional SSO or real password recovery.

7. Student Portal

Dashboard: greeting, date, upcoming response, meal/date/menu preview, deadline, response status, Respond Now action, four meal cards, weekly menu preview, notifications and response history. Statuses include Response Required, Responded, Deadline Passed, Upcoming and Meal Completed.

Meal response: show meal name/date, correct menu, deadline and three large options: Definitely Eating, Probably Eating, Not Eating. Use enum values DEFINITELY_EATING, PROBABLY_EATING, NOT_EATING. One current response per student/meal instance. Resubmission before deadline updates the existing response. After deadline disable choices/submission and show the saved response if any. Use an injectable clock for tests. Never show other students' individual responses.

Weekly menu: show seven days and Breakfast, Lunch, Snacks, Dinner. Support day/week selection and mobile layouts. Clearly label alternating-week alternatives; do not imply all alternatives are served simultaneously.

Notifications: mock in-app notifications with title, message, timestamp, read/unread state and meal link. Implement mark-read and mark-all-read. Do not claim real WhatsApp messages are sent.

History/profile: show only the current student's meal, date, response, update time and deadline status. Basic demo profile and notification preferences; WhatsApp opt-in is only a mock preference.

8. Real Weekly Menu

Use the source file **IIIT DWD MESS MENU – MESS 1.pdf**. If unavailable to the coding agent, request it or clearly document that the transcription needs verification. Do not invent dishes or silently change meaning. Preserve source wording where uncertain and flag it for review.

Preserve source rules: slash-separated items represent alternatives served in alternating weeks; banana limit maximum two pieces; sweets and punugulu have listed limits; fruit limit is one per menu according to the document; ice cream limit one cup; afternoon eggs limited to two. Verify exact wording against the source.

Create typed menu data separate from UI components. Each meal should include stable ID, weekday, meal type, dish list, optional accompaniments, alternative groups, menu notes, and source/verification note. Add dietary/allergen notes only when supported by an authoritative source. Use one source of truth across student/vendor/admin views.

9. Mess Vendor Portal

Design for non-technical users: **one screen, one decision—how much food should we prepare?** Use plain language, prominent quantities, simple statuses and large controls.

Dashboard: four meal cards with date, menu summary, deadline/status and preparation-plan status. Show final response counts only after the deadline. Then display Definitely Eating, Probably Eating, Not Eating, No Response and total population. No Response must not be counted as Not Eating.

Recommendation: show baseline attendance, expected attendance, recommended portions, safety margin, plain-language explanation, and data-quality warning. Mock example: baseline 700 with illustrative 8% margin gives 756 portions. Label this as an example, not a validated optimum. Never imply accuracy just because a number is precise.

Plan: show recommendation, editable quantity, Accept Recommendation, Confirm Plan and optional override reason. Validate quantity. Preserve original recommendation and final vendor quantity separately; confirm before saving.

Outcome: record actual attendance, actual quantity prepared, leftover quantity, explicit unit and optional notes. Do not mix attendance, portions, ingredient weight and leftover weight. Allow leftovers to remain unrecorded if not measured.

10. Administrator Portal

Dashboard: demo counts for students, vendors, admins, configured schedules, upcoming deadlines and menu publication status; quick actions for users, menu, schedules and reports. Clearly mark mock values.

User management: name, demo institute ID/email, role, status, search, filters, add/edit forms and activate/deactivate. Never expose passwords. Confirm sensitive changes. Do not claim institutional identity verification.

Menu management: browse seven days and four meals; edit/add/remove dishes in mock state; manage alternatives and notes; preview student menu; save drafts, publish, cancel/discard edits. Clearly distinguish drafts and published versions.

Menu versioning/persistence: maintain separate draft and published versions. Retain previous published versions for review. Persist drafts, published menu, and version history across page reloads using localStorage in this frontend phase. When publishing, update the shared menu shown to student and vendor views. Preserve the menu version associated with existing meal instances where applicable; changing a future weekly template must not rewrite historical meal menus. Clearly state this is browser-local mock persistence, not server persistence.

Notification schedules: 7 days × 4 meal types = 28 configurable schedules. Each has weekday, meal type, send time, response deadline/duration and enabled state. Use Asia/Kolkata. Breakfast proposal: next-day breakfast notification at 9:00 PM previous evening, deadline 10:00 PM. Lunch after breakfast, snacks after lunch, dinner after snacks. Later meal times remain configurable—not fixed assumptions.

Schedule validation: show meal date separately from notification send date. Validate deadline order, preparation lead time, duplicate/conflicting schedules and disabled status. Detect overlapping response windows that affect the same student. Show a warning listing the affected meals and time windows; do not silently change or automatically reject every overlap, because an administrator may intentionally allow one. Require the administrator to review/acknowledge the warning before saving an overlapping schedule. The backend will later enforce authoritative schedule rules.

Reports: illustrative charts for response coverage, forecast vs actual attendance, planned vs actual prepared quantity, recorded leftovers and forecast error. Label mock data. Show empty states when real data is absent. Do not claim waste reduction without comparable measurements.

Settings: institution display name, timezone, default response duration, mock safety margin and WhatsApp status. Show “Not Connected — Prototype.”

11. Mock Data and Services

Create demo users, seven-day menu, four meal types, dated meal instances, responses, notifications, baseline attendance, vendor decisions, optional outcomes and illustrative reports. Distinguish source-derived values from mock values.

Implement typed service modules: AuthService (login/logout/current user/session); MenuService (weekly/meal menu, save draft, publish); ResponseService (my response, submit/update, history); NotificationService (list/read/mark all); ScheduleService (list/update/validate); ForecastService (vendor summary/recommendation/save decision); OutcomeService (save/list outcomes); ReportService (dashboard and reports). Components should depend on interfaces, not mock implementations.

Use localStorage only where useful for mock session, responses, notification read state, vendor plans, menu drafts/published versions/history and schedule edits. Separate seed data from changes and provide a development-only reset. Never store real passwords/secrets or present localStorage as secure authentication.

12. Domain Types

Define types for User (id, displayName, instituteId/email, role, status); UserRole (STUDENT, VENDOR, ADMIN); MealType (BREAKFAST, LUNCH, SNACKS, DINNER); MealInstance (id, date, mealType, menuVersionId, responseDeadline, status); MealResponse (id, studentId, mealInstanceId, choice, createdAt, updatedAt); ResponseChoice; MealCounts (definitelyEating, probablyEating, notEating, noResponse, totalStudents); ForecastRecommendation (mealInstanceId, baselineAttendance, expectedAttendance, safetyMargin, recommendedPortions, method, dataQualityStatus); VendorDecision (mealInstanceId, originalRecommendation, finalPlannedQuantity, overrideReason, status, updatedAt); MealOutcome (mealInstanceId, actualAttendance, actualPreparedQuantity, leftoverQuantity, leftoverUnit, notes); Notification; NotificationSchedule.

Document date-only values, timestamps and timezone-aware deadlines unambiguously.

13. UX and Interaction Requirements

Every visible action must work in the prototype or be clearly marked as not implemented. Implement navigation, login/logout, role redirects, validation, search/filter, tabs, dialogs, response submission/editing, deadline locking, notification read state, menu draft/publication, schedule editing, recommendation display, vendor overrides, outcome entry, report filters, loading/error/empty/success states and destructive-action confirmations.

Use toast messages where helpful, but do not rely on transient toasts alone for important saved/confirmed state. Never use dead primary buttons. If backend-dependent, show an honest prototype state.

14. Security and Data Integrity

Keep role-specific views separate; validate inputs; never expose another student’s history; do not treat route guards as real security; do not hardcode secrets; do not claim institutional authentication, real WhatsApp delivery or server persistence. Prevent duplicate current responses per student/meal instance in mock data. Keep recommendation, final planned quantity, actual prepared quantity, actual attendance and leftovers separate, with explicit units. Treat no response as its own category.

15. Testing

Write and run tests for: student/vendor/admin login, invalid login, redirects, restricted routes and logout; menu rendering; response submission/update and deadline lock; private response history; admin menu editing and draft/published state; published menu updates; menu/version history persistence after reload; preserving historical meal menu versions; schedule validation including overlapping response windows; user filters; separate No Response count; recommendation display; quantity validation; preserving original recommendation after override; outcome values/units; and empty/error states. Use an injectable clock. Fix TypeScript errors and failing tests before completion.

16. Visual Quality

Make this a polished modern product, not an unstyled admin template, raw forms, cluttered chart dashboard or notebook. Use consistent spacing, clear hierarchy, purposeful icons, thoughtful empty states, responsive navigation, accessible forms, subtle transitions and coherent button variants. Student pages should feel friendly; vendor pages practical; admin pages organised and information-dense without clutter; login welcoming.

17. Development Workflow

1. Inspect existing directory; do not overwrite without inspection. 2. Scaffold Vite/React/TypeScript and configure styles, routing, tests and lint. 3. Define domain types and mock menu data. 4. Build shared components/layouts. 5. Implement login and role routing. 6. Implement student portal. 7. Implement vendor portal. 8. Implement admin portal. 9. Connect mock services and cross-page shared state. 10. Run type checking, lint, tests, production build and inspect desktop/mobile. 11. Document setup, commands, demo accounts, limitations and reset procedure.

18. Definition of Done

Complete only when the app runs locally; all three demo roles work; role routing works; all required pages exist; source menu is transcribed and uncertain entries flagged; student responses can be submitted/updated before deadline and are blocked after it; notifications work in mock state; published menu changes propagate and versions persist across reloads; schedule editing works with overlap warnings; vendor sees final counts including No Response; vendor can accept/override recommendations and record outcomes; reports label demo data; app is responsive/accessibility-conscious; primary actions work; type checking, lint, tests and production build pass; setup is documented; and no real backend/WhatsApp/institutional authentication is falsely claimed.

19. Final Implementation Report

After implementation, report: what was built; folder structure; installation/run commands; demo credentials for each role; actual test/build results; menu entries needing verification; simulated versus connected features; known issues; and recommended backend next steps. Do not claim completion if tests fail or required screens are missing. Begin by inspecting the project directory, then implement the application.

Operational Note

If the coding agent cannot access the conversation upload, place the original menu PDF in docs/IIIT_DWD_MESS_MENU_MESS_1.pdf and instruct the agent to use it as the source of truth. Provide the architecture/formula PDF if desired. This phase must not implement a real backend.